Page 1: Purpose and learning goals

The RAG Learning Assistant is a source-grounded study system. It helps a learner work with a personal collection of documents while keeping every generated statement traceable to stored source passages. Retrieval-augmented generation separates finding evidence from writing an answer. The retrieval component selects relevant chunks, and the generation component receives those chunks as untrusted source material. This boundary matters because document text can contain mistakes, misleading instructions, or content that was never intended to control the application.

The project is also a learning environment for software architecture. Each responsibility has a narrow interface, persistent formats are validated, and tests describe behavior before implementation. A useful system must do more than produce fluent text. It must preserve document identity, page numbers, chunk order, model revisions, prompt versions, and citation relationships. These details make an answer inspectable and allow later experiments to compare results fairly.

The benchmark document is synthetic. It contains no external book text and may be redistributed under the repository license. Its ten stable pages provide enough material to exercise extraction, chunking, embedding, persistent indexing, map summaries, reduction, citations, caching, and performance measurement.

Page 2: Document ingestion

Ingestion converts a PDF into application-owned data. The extractor opens the file, visits pages in their original order, and records text together with a one-based page number and source filename. Keeping page boundaries is essential because citations must lead a learner back to the exact place where supporting information appeared. An extraction adapter hides library-specific objects so the application does not depend directly on every detail of the PDF library.

PDF text extraction is not optical character recognition. A scanned page may contain only an image and therefore produce no useful text. Fonts and encodings can also lead to garbled characters even when a page looks correct in a viewer. Production ingestion should eventually report these quality problems rather than silently indexing unusable content. The current benchmark deliberately uses ordinary embedded text so it measures summarization instead of OCR behavior.

The extractor creates immutable page models. Validation rejects blank source names and invalid page numbers close to the boundary where data enters the system. Later stages can therefore assume that page identity is meaningful. Resource handling is explicit: PDF handles are closed after extraction, including when processing raises an exception.

Page 3: Chunking and overlap

Language models and embedding models cannot consume an unlimited document in one request. Chunking divides extracted pages into smaller passages while retaining the source, page number, and a document-wide index. The current implementation begins with a character budget because it is easy to understand and deterministic across model providers. A future token-aware strategy can estimate model cost more accurately.

Adjacent chunks overlap so a sentence or explanation near a boundary is not separated from all of its context. Without overlap, a query about a concept split across two chunks may match neither fragment strongly. Too much overlap has a cost: duplicated text increases embedding work, index size, retrieval redundancy, and summarization input. The chosen overlap is therefore a compromise rather than a universally correct value.

Chunk indices are continuous across the document. Page numbers identify the human-visible source location, while chunk indices distinguish multiple passages from the same page. A stable document UUID connects chunks to the catalog entry even when two documents share the same filename. These identifiers later allow removal and replacement without disturbing unrelated documents.

Page 4: Embeddings and retrieval

An embedder transforms text into a fixed-length numerical vector. The project uses a pinned multilingual E5 model whose vectors contain 384 dimensions. A dimension is not a word slot. Together, all dimensions encode learned semantic features, allowing passages with related meaning to be close even when they use different words. Document passages and search queries use different E5 prefixes because the model was trained with distinct roles for candidates and queries.

FAISS performs efficient similarity search over vectors. The search result score describes vector similarity, but it does not prove that a passage is correct or sufficient to answer a question. Retrieval returns ranked candidates together with their original chunks. The application limits the number of results so generation receives focused evidence instead of an uncontrolled amount of context.

Retrieval quality depends on chunk boundaries, embedding model, query wording, and document quality. A fluent generator cannot repair missing evidence reliably. Evaluation should therefore inspect retrieval separately from answer quality. Useful checks include whether the expected passage appears in the top results and whether irrelevant passages are ranked below it.

Page 5: Persistent storage

The library uses FAISS and SQLite because they solve different persistence problems. FAISS stores vectors and searches them efficiently. SQLite stores structured metadata such as documents, chunk text, page numbers, model identity, and mappings between FAISS positions and chunks. Keeping metadata outside the vector index makes relationships queryable and allows the application to reconstruct trusted citations after a search.

Every indexed document receives a UUID and a SHA-256 content digest. The UUID remains stable when a document is deliberately replaced, while the digest detects duplicate content and distinguishes revisions. Model name, pinned revision, and vector dimension are stored with the index so incompatible embeddings cannot be mixed accidentally.

Persistent updates require careful ordering. Adding, removing, or replacing a document affects both stores. A failure between operations can leave inconsistent state unless the workflow validates work before committing and provides recovery. The current architecture keeps these responsibilities behind repository and vector-store interfaces, leaving room for stronger transactional recovery later.

Page 6: The document library

The library application service coordinates document management rather than implementing PDF extraction or vector search itself. Adding a document checks for duplicate content, extracts pages, creates chunks, indexes embeddings, and registers catalog metadata. Listing returns stable document identities. Removal deletes only data associated with the selected UUID. Replacement validates and prepares new content before preserving the existing identity with updated metadata.

Protocols describe the behavior required from collaborators. A concrete SQLite repository does not need to inherit from a base class when it structurally provides the required methods. This keeps application code independent of one persistence technology and makes focused test doubles possible. Tests can record calls and simulate failures without loading machine-learning models.

Batch import processes multiple paths sequentially and reports each outcome. Duplicate documents are skipped, ordinary failures are recorded, and a problem with one path does not prevent later inputs from being attempted. This behavior supports practical library maintenance while keeping the result machine-readable for scripts and user interfaces.

Page 7: Grounded generation

Generation turns retrieved evidence into a readable answer. The application builds a prompt that explicitly forbids unsupported prior knowledge, marks contexts as untrusted data, and requests citations by context number. The Hugging Face adapter adds a system instruction requiring strict JSON. The model returns answer text and context numbers, while the application reconstructs source, page, chunk index, and excerpt from trusted stored data.

Prompt instructions influence behavior but do not guarantee compliance. The response parser rejects malformed JSON, unexpected fields, invalid types, duplicate citations, and non-positive numbers. A small local model receives exactly one repair attempt when formatting is invalid. The repair prompt permits correction of representation only; it does not authorize new facts or citations.

Prompt templates have names, explicit versions, and SHA-256 fingerprints. Runtime results expose compact prompt references so an experiment can identify which instructions were used. Changing meaningful prompt text requires a version change. This discipline supports comparisons and prevents cached results from crossing incompatible prompt configurations.

Page 8: Document-wide summarization

A focused retrieval query cannot represent every important part of a long document. Document-wide summarization therefore reads all stored chunks in order and uses a map-reduce workflow. The map phase groups consecutive chunks within a conservative input budget and creates a grounded partial summary for each group. Each partial result may cite only context numbers belonging to its own batch.

The reduce phase combines partial summaries into one concise result. It may use only original context numbers already supporting at least one partial summary. Section order is deliberately not represented as a number because a small model might confuse a section label with a citation. The final citation metadata is reconstructed from the complete ordered chunk list.

Character-based batching is simple but only approximates token cost. Prompt instructions, XML-style boundaries, source labels, and citation metadata add characters beyond the configured chunk-text budget. Very large inputs can increase attention cost and GPU memory sharply. Benchmarking different budgets is necessary before changing defaults.

Page 9: Resumable summaries and identity

Long summaries can require many GPU calls. Repeating every completed map batch after an interruption wastes time and energy. The summary cache stores each validated map result in the library's SQLite database. A later invocation looks up batches before generation and continues with the first missing result. Cache hits skip both model work and the corresponding progress message.

Reuse is safe only when all influential inputs match. A generation identity includes document content SHA-256, model name and pinned revision, prompt references, maximum generated tokens, and the batch character budget. Canonical JSON and SHA-256 produce a stable fingerprint across processes. Stored context-number ranges detect a changed batch plan even if a programming error reused a key.

Cache writes are immutable and idempotent. Writing identical data again is harmless, but conflicting data under the same identity and batch number is rejected. Generated citations are validated before persistence so malformed output cannot poison future resume attempts. The final reduction is currently regenerated on every invocation because map calls dominate the resumability benefit.

Page 10: Measurement and future work

A useful benchmark records configuration and environment together with time. This project measures every successful map and reduce call, input and output character counts, total runtime, cache decisions, GPU model, and peak allocated GPU memory. The first generation includes lazy model initialization. Cache-free runs measure raw generation, while normal runs show the practical benefit of resuming completed work.

One run is not enough to establish a universal default. Operating-system file caches, runtime initialization, generated length, dependency versions, and background workload introduce variation. Repeated trials should report a median and spread. Summary quality must also be reviewed because the fastest configuration is not useful if it omits central facts or produces unreliable citations.

Near-term optimization candidates include separate token limits for map and reduce, token-aware input batching, compact prompt metadata, and evaluation of smaller or quantized models. Parallel generation is risky on an eight-gigabyte GPU when a single large batch nearly fills memory. Changes should be measured against this stable, redistributable fixture and recorded without presenting hardware-specific observations as architectural guarantees.